Linnæus University
Sweden

Degree project

Linear algebra in computer graphics

Author: Ana Strikic
Supervisor: Per Anders Svensson
Examiner: Hans Frisk
Date: 2019-09-17
Course Code: 2MA41E
Subject: Mathematics
Level: Bachelor

Department Of Mathematics

Abstract

In this thesis, we will investigate the rapidly developing field of computer graphics by giving an insight into the calculations behind the most important topics in perspectives, shading and distance. We will delve into frequently used shading algorithms and perspective transforms in areas of computer science, architecture and photography, such as pseudo fish-eye lenses and wide-angle lenses.

Contents

1 Introduction **1**

2 Methods **3**

2.1 Perspectives 3

2.1.1 Projecting an object onto a forward projection plane 3

2.1.2 Vertically oblique projection 4

2.1.3 Horizontally oblique projection 6

2.2 Lighting and shading 8

2.2.1 Light sources and Lambertian reflectance 8

2.2.2 Light reflection 9

2.2.3 Flat shading 11

2.3 Distances between objects 12

2.3.1 Closest distance between a stationary observer and a moving object 12

3 Results **15**

3.1 Perspectives 15

3.1.1 Arbitrary oblique projection 15

3.1.2 Fish-eye perspective 18

3.1.3 Wide angle perspective 19

3.2 Lighting and shading 20

3.2.1 Gouraud shading 20

3.2.2 Phong shading 21

3.2.3 Bump mapping 22

3.3 Distances between objects 24

3.3.1 Closest distance between a moving observer and a moving object 24

4 Discussion **27**

References **31**

1

Introduction

Computer graphics is currently a very fast developing topic in technology, with more realistic ways to depict the world being found rapidly. In this thesis, we will attempt to uniformly present most important knowledge and calculations behind the commonly used algorithms in computer graphics.

Firstly, the thesis will focus on perspective changes, covering arbitrary projection planes and special lenses, such as fish-eye and wide angle. These calculations will provide algorithms to, given a point in space, locate the placement of the pixel representing that point on a computer screen, once it has undergone a perspective change.

Secondly, the thesis will consider the most commonly used shading algorithms, as well as their drawbacks and advantages, along with examples. In this part, we will also look at the way light reflects and the frequent considerations taken into account when representing a light reflection.

Lastly, we will briefly calculate the shortest distance between a moving object and an observer, in cases of both a moving and a stationary observer.

As this knowledge is quite dispersed and seldom taught with much explaination, the intention of this thesis is to compile the calculations, as well as, hopefully, provide a useful insight for further developments in this field.

2

Methods

2.1 Perspectives

To fully understand how various projections are achieved, the knowledge of basic transformations and projections is necessary. Therefore, this section will focus on vertically and horizontally oblique perspective planes. We let $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ be the standard basis.

2.1.1 Projecting an object onto a forward projection plane

Letting the origin of the coordinate system be an observer and their view field the projection plane, it is quite elementary to achieve the most basic, forward projection with respect to the observer.

Without the loss of generality, assume that the observer is looking in the direction of the z-axis and thus, the view field π to be parallel to the xy-plane at a distance d from the origin. Any object with coordinates $P(x, y, z)$ can be, with the help of matrix transformations, projected onto a point $P'(x', y') \in \pi$ necessarily colinear with P and the origin O.

As $z' = d$, the task is to find x', y'. By simple vector addition, it is noticable that

$$\overrightarrow{OP'} = \mathbf{i}x' + \mathbf{j}y' + \mathbf{k}d$$

or

$$\overrightarrow{OP'} - \mathbf{k}d = \mathbf{i}x' + \mathbf{j}y'. \tag{2.1}$$

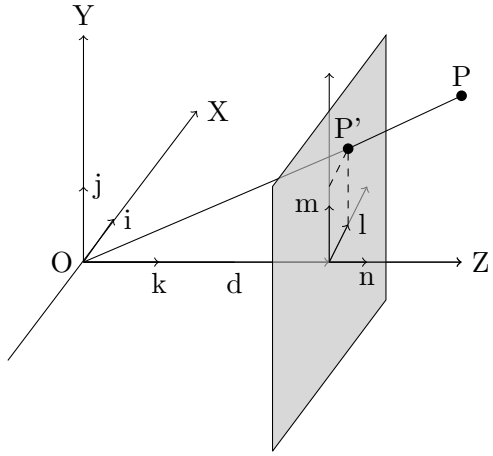


Figure 2.1: Graphical representation of the forward projection

As O, P, P' are colinear, $\overrightarrow{OP'}$ can be expressed as a scaled version of $\overrightarrow{OP}$, i.e.

$$\overrightarrow{OP'} = \lambda \cdot \overrightarrow{OP}$$

where $\lambda \in \mathbb{R}$.

Projections on the Z-axis of $\overrightarrow{OP'}$ and $\lambda \cdot \overrightarrow{OP}$ are clearly, identical. Therefore it is possible to write λ as a ratio between the projections on $\mathbf{k}$.

$$\lambda = \frac{d}{z}. \tag{2.2}$$

By (2.1) and (2.2) it is possible to derive

$$\mathbf{i}x' + \mathbf{j}y' = \frac{d}{z}(x\mathbf{i} + y\mathbf{j} + z\mathbf{k}) - d\mathbf{k}.$$

This equation leads to general projection plane coordinates for y, z

$$x' = d\frac{x}{z}, \qquad y' = d\frac{y}{z}.$$

2.1.2 Vertically oblique projection

For the further calculations of projections, the new axial system achieved by the projection plane is denoted with vectors $\mathbf{l}$, $\mathbf{m}$, and $\mathbf{n}$.

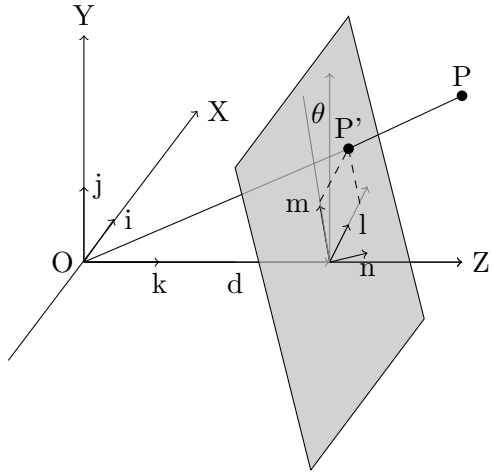


Figure 2.2: Graphical representation of the vertically oblique projection

To obtain a vertically oblique projection, the projection plane is rotated about the l-axis, parallel to the x-axis, for angle θ.

Using matrix transformations provides a relationship between the two axial systems

$$\begin{bmatrix} \mathbf{l} \\ \mathbf{m} \\ \mathbf{n} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} \mathbf{i} \\ \mathbf{j} \\ \mathbf{k} \end{bmatrix}.$$

Performing the same calculations as in the basic projection, leads to similar results

$$\overrightarrow{OP'} = \mathbf{l}x' + \mathbf{m}y' + d\mathbf{k}$$

or

$$\overrightarrow{OP'} - d\mathbf{k} = \mathbf{l}x' + \mathbf{m}y'. \tag{2.3}$$

As O, P, P' are colinear, $\overrightarrow{OP'}$ can, again, be expressed as a scaled version of $\overrightarrow{OP}$, i.e.

$$\overrightarrow{OP'} = \lambda \cdot \overrightarrow{OP}$$

where $\lambda \in \mathbb{R}$.

By similar reasoning to the previous case, projections on the $\mathbf{n}$ of $\overrightarrow{OP'}$ and $\lambda \cdot \overrightarrow{OP}$ are identical. Hence it is possible to write λ as a ratio between the projections on $\mathbf{n}$

$$\lambda = \frac{d\mathbf{n} \cdot \mathbf{k}}{\mathbf{n} \cdot \overrightarrow{OP}}.$$

It is now neccessary to express $\mathbf{n}$ in terms of the original axial system, which produces

$$\mathbf{n} = \sin\theta\mathbf{j} + \cos\theta\mathbf{k}$$

and

$$\lambda = \frac{d\cos\theta}{y\sin\theta + z\cos\theta}.$$

Expressing $\mathbf{m}$ in terms of the original axial system and expanding (2.3) gives the equations for obtaining x' and y'.

$$x' = \lambda x = \frac{dx\cos\theta}{y\sin\theta + z\cos\theta},$$

$$y' = \frac{\lambda y}{\cos\theta} = \frac{dy}{y\sin\theta + z\cos\theta}.$$

2.1.3 Horizontally oblique projection

Equations regarding x' and y' in horizontally oblique projections are achieved in like manner to the vertically oblique projection.

Using the transformation matrix

$$\begin{bmatrix} \mathbf{l} \\ \mathbf{m} \\ \mathbf{n} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \cdot \begin{bmatrix} \mathbf{i} \\ \mathbf{j} \\ \mathbf{k} \end{bmatrix}.$$

one can relate the two axial systems.

This produces relation equations for $\mathbf{l}$, $\mathbf{m}$ and $\mathbf{n}$ with regards to the original system as follows

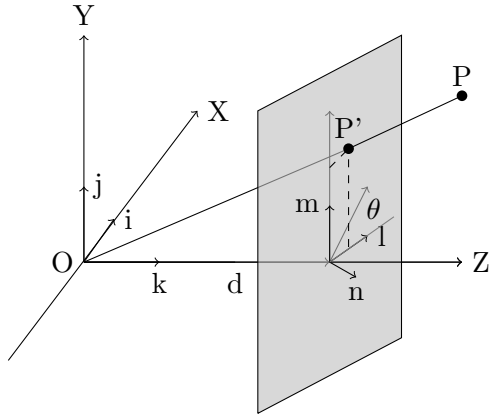


Figure 2.3: Graphical representation of the horizontally oblique projection

$$\mathbf{l} = \cos\theta\mathbf{i} + \sin\theta\mathbf{k}$$

$$\mathbf{m} = \mathbf{j}$$

$$\mathbf{n} = -\sin\theta\mathbf{i} + \cos\theta\mathbf{k}$$

By the same reasoning as with vertically oblique projections, λ is achieved from two identical projections on $\mathbf{n}$,

$$\lambda = \frac{d\mathbf{n}\cdot\mathbf{k}}{\mathbf{n}\cdot\overrightarrow{OP}} = \frac{d\cos\theta}{-x\sin\theta + z\cos\theta}.$$

The equation

$$x'\mathbf{l} + y'\mathbf{m} = \overrightarrow{OP'} - d\mathbf{k}$$

therefore, becomes

$$x'(\cos\theta\mathbf{i} + \sin\theta\mathbf{k}) + y'\mathbf{j} = \lambda(x\mathbf{i} + y\mathbf{j} + z\mathbf{k}) - d\mathbf{k}. \tag{2.4}$$

Solving (2.4) for x' and y' gives the final equations

$$x' = \frac{\lambda x}{\cos\theta} = \frac{dx}{-x\sin\theta + z\cos\theta},$$
$$y' = \lambda y = \frac{dy\cos\theta}{-x\sin\theta + z\cos\theta}.$$

Reasonably, when $\theta = 0$ in either of the oblique projections, the projection becomes the forward projection covered in the first part the section.

2.2 Lighting and shading

2.2.1 Light sources and Lambertian reflectance

Light sources are most commonly divided into three types; point, directional and spotlight. A point light provides light in all directions from one point in the space, a directional light provides a parallel beam of light rays, while a spotlight behaves as a conical beam. Most commonly used light source is point light which is going to be further investigated in this thesis.

Light at a point P is represented by a vector originating at the illuminated point P and ending at the source point L

$$\mathbf{L} = \begin{bmatrix} L_x - P_x \\ L_y - P_y \\ L_z - P_z \end{bmatrix},$$

where $L = (L_x, L_y, L_z), P = (P_x, P_y, P_z)$.

Using this vector it is possible to determine how much light is illuminating a certain surface at every point. It is assumed that the sufrace causes diffuse reflection, or in other words, causes all light hitting it to reflect back equally in every direction.

To find the amount of light, E, that is hitting a line CB on the surface, we observe the orthogonal projection AB of CB on a surface perpendicular to the light rays. It is clear that

$$E \propto \frac{AB}{CB},$$
$$\frac{AB}{CB} = \cos\theta.$$

This equation can be written with the help of light vector $\mathbf{L}$ and the normal vector of the plane that is being illuminated, $\mathbf{n}$, as

$$\cos\theta = \frac{\mathbf{n} \cdot \mathbf{L}}{||\mathbf{n}|| \cdot ||\mathbf{L}||}.$$

Letting $\mathbf{n}$ and $\mathbf{L}$ be unit vectors leads to the standard equation of surface light intensity I on a surface with incident light intensity I_i

$$I = I_i \mathbf{n} \cdot \mathbf{L}.$$

Neglecting specular reflection as in this case, leads to a formula for an ideally matte surface called Lambertian surface, after Johann Heinrich Lambert, who first introduced such a surface in his book Photometria, published in 1760.

2.2.2 Light reflection

As in real world, illumination of a certain object is caused, not only by the light source but also, by all the objects close by reflecting some of the light that is hitting them. This reflection is divided into three different types, ambient, diffuse and specular.

k_a - ambient reflection which represents a constant level of light in the space (i.e. sunlight, moonlight

k_d - diffuse reflection which represents light reflected off of matte surfaces

k_s - specular reflection which represents light reflected off of shiny surfaces

These three types of reflections encompass all of the reflected light, thus it is clear that

$$k_a + k_d + k_s = 1$$

holds.

Defining ambient light level as I_a, and the incident light as previously, I_i, it is possible to write ambient and diffuse components of light as

$$k_a I_a \quad \text{and}$$
$$k_d I_i \mathbf{n} \cdot \mathbf{L} \quad \text{, respectively.}$$

Finally, to calculate the specular term one has to look at the geometry of light relfection. If a light ray, with direction **-L**, enters a point P at an angle θ to the surface normal **n** then it reflects in a direction **R** such that is at an angle $-\theta$ to the surface normal at P.

For an observer looking at direction **R**, the surface has a bright spot, however, for an observer that is looking in direction **V**, offset by angle α, while the surface does not have a perfectly bright spot, it does reflect some light.

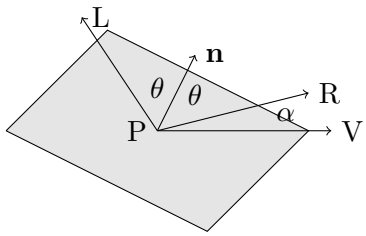


Figure 2.4: Reflection and lighting at point P

B.T. Phong proposed the following equation

$$\cos\alpha = \mathbf{R} \cdot \mathbf{V}. \tag{2.5}$$

which produces a bell-shaped illumination around point P when viewed at an angle α. Since all surfaces have a different level of shininess, i.e. a metal is not as shiny as a diamond, Phong also proposed that raising equation (2.5) to a power n would produce the different shininess levels (Pho75). Therefore

$$\cos^{1000}\alpha = (\mathbf{R} \cdot \mathbf{V})^{1000},$$
$$\cos^{30}\alpha = (\mathbf{R} \cdot \mathbf{V})^{30},$$

could, possibly, represent a shiny metal sword and a character's face illuminated by the sun, respectively. The exponent n is called the *Phong exponent*.

Combining the specular component with the ambient and diffuse components leads to

$$I = k_a I_a + I_i(k_d \mathbf{n} \cdot \mathbf{L} + k_s(\mathbf{R} \cdot \mathbf{V})^n). \tag{2.6}$$

Equation (2.6), however, describes only the amount of light, i.e. the intensity, therefore neglecting colour. As lights can be of any colour in the visible colour spectrum, this equation needs to be refined to accomodate for colour as well as intensity. The standard red, green, blue colour system (RGB) is used by dividing equation (2.6) into a system of three similar RGB equations as follows

$$\begin{cases} I_r = & k_{ar} I_{ar} + I_{ir}(k_{dr} \mathbf{n} \cdot \mathbf{L} + k_{sr}(\mathbf{R} \cdot \mathbf{V})^n) \\ I_g = & k_{ag} I_{ag} + I_{ig}(k_{dg} \mathbf{n} \cdot \mathbf{L} + k_{sg}(\mathbf{R} \cdot \mathbf{V})^n) \\ I_b = & k_{ab} I_{ab} + I_{ib}(k_{db} \mathbf{n} \cdot \mathbf{L} + k_{sb}(\mathbf{R} \cdot \mathbf{V})^n). \end{cases}$$

Still, vector $\mathbf{R}$ is yet to be calculated.

In 1978, James Blinn proposed a way to avoid calculating $\mathbf{R}$ althogether by taking the vector $\mathbf{h}$ as the halfway between the light direction $\mathbf{L}$ and the observer direction $\mathbf{V}$ (Bli78)

$$\mathbf{h} = \frac{\mathbf{L}+\mathbf{V}}{||\mathbf{L}+\mathbf{V}||}.$$

With this new vector, $\mathbf{h}$, $\mathbf{R} \cdot \mathbf{V}$ can then be replaced with $\mathbf{n} \cdot \mathbf{h}$, producing the final equation of the Blinn-Phong shading model

$$I = k_a I_a + I_i(k_d \mathbf{n} \cdot \mathbf{L} + k_s(\mathbf{n} \cdot \mathbf{h})^n), \tag{2.7}$$

which is currently in use in most of the action programming interfaces (APIs).

2.2.3 Flat shading

Flat shading is the simplest and most time effective way of rendering shadows. However, due to its simplicity, this model leads to very unrealistically looking shadows.

The algorithm takes a single point on a polygon and calculates the light intensity on it. The whole polygon is then rendered with the same light intensity. In some engines,

the algorithm also calculates the light intensity on the vertices, however it still produces a quite blocky and flat look of the models.

Because of its simplicity, this algorithm requires close to no mathematical correction, but it is still a good baseline for more precise algorithms, such as Gouraud and Phong shading algorithms.

2.3 Distances between objects

When an object moves towards an observer, from the observers perspective, that object becomes larger and larger until it reaches the closest point to the observer. Therefore, to be able to realistically depict size and closeness of the object, it is very important to be able to find the point of *close encounter* as well as the time and the distance between the object and observer at that point. These close encounter points are divided into two kinds; close encounter of the first kind and close encounter of the second kind, first of which happens between a stationary observer and a moving object, and second of which is between a moving observer and a moving object (Vin07).

2.3.1 Closest distance between a stationary observer and a moving object

For an observer stationed at point P and an object moving from point Q to Q', the closest point C between the observer and the object is achieved when $\overrightarrow{PC} \perp \overrightarrow{QQ'}$.

Denoting $\overrightarrow{OC} = \mathbf{c}, \overrightarrow{OP} = \mathbf{p}, \overrightarrow{PC} = \mathbf{d}, \frac{\overrightarrow{QQ'}}{|\overrightarrow{QQ'}|} = \mathbf{v}$ and $\overrightarrow{OQ} = \mathbf{q}$ it is possible to write, for some point K on object's path,

$$\overrightarrow{OK} = \mathbf{q} + s\mathbf{v}t_k,$$

where t_k represents time required for the object to reach point K and s the speed of the object. Thus,

$$\mathbf{c} = \mathbf{q} + s\mathbf{v}t_c. \tag{2.8}$$

As $\mathbf{d} \perp \mathbf{v}$, it follows that

$$\mathbf{v} \cdot \mathbf{c} = \mathbf{v} \cdot \mathbf{p}.$$

.

Multiplying (2.8) by $\mathbf{v}$, one obtains

$$\mathbf{v} \cdot \mathbf{c} = \mathbf{v} \cdot \mathbf{q} + st_c \mathbf{v} \cdot \mathbf{v} \Leftrightarrow \mathbf{v} \cdot \mathbf{p} = \mathbf{v} \cdot \mathbf{q} + st_c.$$

This produces the equations for the time and distance of the *close encounter* point,

$$||\mathbf{d}|| = ||\mathbf{c} - \mathbf{p}|| = ||\mathbf{q} + t_c s\mathbf{v} - \mathbf{p}||,$$

$$t_C = \frac{\mathbf{v} \cdot (\mathbf{p} - \mathbf{q})}{s}.$$

If the observer is positioned at the origin, as they usually are, then the equations become

$$||\mathbf{d}|| = ||\mathbf{c} - \mathbf{p}|| = ||\mathbf{q} + t_c s\mathbf{v}||, \tag{2.9}$$

$$\text{and } t_C = \frac{-\mathbf{v} \cdot \mathbf{q}}{s}. \tag{2.10}$$

3

Results

3.1 Perspectives

3.1.1 Arbitrary oblique projection

Since vertically and horizontally oblique projections have already been presented in the methods, one can simply derive the transformation and results for an arbitrary oblique projection, in which the projection plane is rotated by angle ϕ around x-axis, angle ψ around y-axis and angle θ around z-axis. These angles are more commonly known as *pitch*, *yaw* and *roll* angles, respectively.

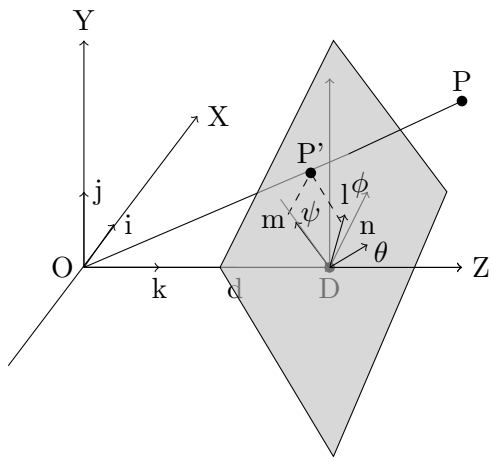


Figure 3.1: Visual representation of an arbitrary oblique projection

Firstly, the transformation matrix between the axial systems is given with

$$\begin{bmatrix} \mathbf{l} \\ \mathbf{m} \\ \mathbf{n} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{i} \\ \mathbf{j} \\ \mathbf{k} \end{bmatrix}.$$

where c_{ij} denotes variables dependent on the roll, pitch and yaw of the view plane. More precisely, using the standard convention of applying yaw, pitch, roll transformations, in that order, the final transformation can be written as:

$$\begin{bmatrix} \mathbf{l} \\ \mathbf{m} \\ \mathbf{n} \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & \sin\phi \\ 0 & -\sin\phi & \cos\phi \end{bmatrix} \cdot \begin{bmatrix} \cos\psi & 0 & -\sin\psi \\ 0 & 1 & 0 \\ \sin\psi & 0 & \cos\psi \end{bmatrix} \cdot \begin{bmatrix} \mathbf{i} \\ \mathbf{j} \\ \mathbf{k} \end{bmatrix}.$$

This leads to formulas for the entries of (c_{ij}).

$$\begin{cases} c_{11} & = \cos\theta\cos\psi + \sin\theta\sin\phi\sin\psi \\ c_{12} & = \sin\theta\cos\phi \\ c_{13} & = -\cos\theta\sin\psi + \sin\theta\sin\phi\cos\psi \\ c_{21} & = -\sin\theta\cos\psi + \cos\theta\sin\phi\sin\psi \\ c_{22} & = \cos\theta\cos\phi \\ c_{23} & = \sin\theta\sin\psi + \cos\theta\sin\phi\cos\psi \\ c_{31} & = \cos\phi\sin\psi \\ c_{32} & = -\sin\phi \\ c_{33} & = \cos\phi\cos\psi. \end{cases}$$

For simplicity, the further calculations will use the variable names c_{ij}.

The final result of the calculations should still be an ordered pair (x', y') of coordinates on the view plane.

Firstly, using the transformation matrix, **lmn** axial system can be written in terms of **ijk** axial system as so

$$\mathbf{l} = c_{11}\mathbf{i} + c_{12}\mathbf{j} + c_{13}\mathbf{k} \tag{3.1}$$

$$\mathbf{m} = c_{21}\mathbf{i} + c_{22}\mathbf{j} + c_{23}\mathbf{k} \tag{3.2}$$

$$\mathbf{n} = c_{31}\mathbf{i} + c_{32}\mathbf{j} + c_{33}\mathbf{k}. \tag{3.3}$$

Similar to vertically and horizontally oblique projections, it holds that

$$\overrightarrow{OP'} = \mathbf{d} + \overrightarrow{DP'},$$
$$\overrightarrow{DP'} = x'\mathbf{l} + y'\mathbf{m}.$$

Therefore,

$$x'\mathbf{l} + y'\mathbf{m} = \overrightarrow{OP'} - \mathbf{d}. \tag{3.4}$$

Defining $\overrightarrow{OP'} = \lambda \cdot \overrightarrow{OP}$ and combining two projections of $\overrightarrow{OP'}, \lambda \cdot \overrightarrow{OP}$ on $\mathbf{n}$ it is possible to find λ as

$$\lambda = \frac{d\mathbf{n} \cdot \mathbf{k}}{\mathbf{n} \cdot \overrightarrow{OP}} = \frac{dc_{33}}{xc_{31} + yc_{32} + zc_{33}}. \tag{3.5}$$

Equation (3.4) now becomes

$$x'\mathbf{l} + y'\mathbf{m} = \lambda \cdot \overrightarrow{OP} - d\mathbf{k}. \tag{3.6}$$

As, $\mathbf{l} \cdot \mathbf{l} = \mathbf{m} \cdot \mathbf{m} = 1$ and $\mathbf{l} \cdot \mathbf{m} = \mathbf{m} \cdot \mathbf{l} = 0$, multiplying (3.6) with $\mathbf{l}$ and $\mathbf{m}$ will lead to equations for x', y', respectively.

Rewritting equation (3.6) then leads to

$$x' = \lambda\mathbf{l} \cdot \overrightarrow{OP} - d\mathbf{l} \cdot \mathbf{k},$$
$$y' = \lambda\mathbf{m} \cdot \overrightarrow{OP} - d\mathbf{m} \cdot \mathbf{k},$$

which when combined with equations (3.1) and (3.2) as well as

$$\overrightarrow{OP} = x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$$

produces the final equations

$$x' = \lambda(xc_{11} + yc_{12} + zc_{13}) - dc_{13}$$
$$y' = \lambda(xc_{21} + yc_{22} + zc_{23}) - dc_{23}.$$

Substituting results of equation (3.5) gives

$$x' = dc_{33}\frac{(xc_{11} + yc_{12} + zc_{13})}{xc_{31} + yc_{32} + zc_{33}} - dc_{13},$$
$$y' = dc_{33}\frac{(xc_{21} + yc_{22} + zc_{23})}{xc_{31} + yc_{32} + zc_{33}} - dc_{13},$$

where variables c_{ij} are given as in system (3.1-3.2).

3.1.2 Fish-eye perspective

So far, all of the projections have been oblique, but projecting with a lens, i.e. onto a spherical or ellipsoidal surface, gives both the fish-eye and wide angle projections, respectively (KMH95). Radius of the sphere is denoted by r and the centre of curvature is the observer in the origin.

Following the similar pattern to oblique projections, the object of projection is point $P = (x, y, z)$ and the point of projection on the lens in $P' = (x', y')$. Third coordinate of the projected point is defined by the equation of the sphere lens.

As before, denote

$$\overrightarrow{OD} = d\mathbf{k}.$$

It is possible to define $\overrightarrow{OP'}$ in terms of $\overrightarrow{OP}$ as

$$\overrightarrow{OP'} = \lambda \cdot \overrightarrow{OP}. \tag{3.7}$$

Since P' lies on the sphere of radius r it is obvious that $||\overrightarrow{OP'}|| = r$ for all points on the lens. Hence,

$$\lambda = \frac{||\overrightarrow{OP'}||}{||\overrightarrow{OP}||} = \frac{r}{||\overrightarrow{OP}||}. \tag{3.8}$$

Equations (3.7) and (3.8) lead to

$$\overrightarrow{OP'} = \frac{r}{||\overrightarrow{OP}||}\overrightarrow{OP},$$

and specifically to

$$x' = \frac{rx}{\sqrt{x^2 + y^2 + z^2}}, \qquad y' = \frac{ry}{\sqrt{x^2 + y^2 + z^2}}.$$

3.1.3 Wide angle perspective

Wide angle perspective leads to many issues when projected onto a spherical surface as it tends to bend straight lines and distort objects. Various approaches have been considered and are being used, however this dissertation will only cover an ellipsoidal lens projection. As previously stated, the ellipsoidal lens can be used to represent a wide angle lens in order to get a perspective that, while not always optimally, is quite flexible at conserving straight lines and object shapes (ZWXH11).

Similarly to fish-eye spherical projection, letting the ellipsoid of the equation

$$a^2x^2 + b^2y^2 + z^2 = R^2 \tag{3.9}$$

be the lens centered around the observer in the origin, leads to equations for $P' = (x', y')$.

Representing $\overrightarrow{OP'}$ as $\lambda \cdot \overrightarrow{OP}$ gives

$$\lambda = \frac{||\overrightarrow{OP'}||}{||\overrightarrow{OP}||} = \frac{R}{||\overrightarrow{OP}||},$$

since the point P' again lies on the ellipsoid with equation 3.9.

By comparable thinking to the fish eye perspective, it is concluded that

$$\overrightarrow{OP'} = \frac{R}{||\overrightarrow{OP}||}\overrightarrow{OP}$$

or specifically

$$x' = \frac{Rax}{\sqrt{a^2x^2 + b^2y^2 + z^2}}, \qquad y' = \frac{Rby}{\sqrt{a^2x^2 + b^2y^2 + z^2}}.$$

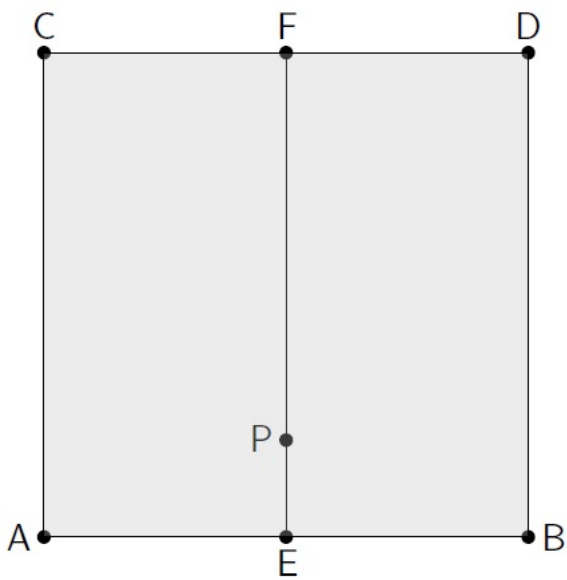
3.2 Lighting and shading

In rendering, objects are separated into smaller planar polygons for simplicity, for example a sphere is divided into a large number of triangles and trapezoids that closely resemble a sphere when connected together. Flat shading these objects would then make curved objects look blocky as every polygon would be single coloured and harsh lines between two polygons would be obvious. To resolve this issue, Henri Gouraud and B. T. Phong have developed two different algorithms that are still in use in computer graphics.

3.2.1 Gouraud shading

Noticing the inadequacies of flat shading, H.Gouraud (Gou71) proposed that instead of taking the polygon surface normal and applying it to the entire polygon, one could use the polygon vertex normals and interpolate the light intensity between them.

Figure 3.2: Polygon $ABCD$ and point P on the scan line EF



Given a polygon $ABCD$ and a scan line (i.e. an arbitrary line containing point P at which we want to calculate shading), if the surface normals are known at points A, B, C and D, then it is possible to calculate the shadings at points E and F, the intersections of the scan line with AB and CD respectively, as

$$
\begin{aligned}
I_E =& (1-\alpha)I_A + \alpha I_B \qquad (0 \leq \alpha \leq 1) \\
I_F =& (1-\beta)I_C + \beta I_D \qquad (0 \leq \beta \leq 1),
\end{aligned}
$$

where α and β are calculated as

$$
\alpha = \frac{|AE|}{|AB|}, \qquad \beta = \frac{|CF|}{|CD|}.
$$

.

Similarly, shading at point P on the scan line EF can be calculated as

$$
I_P = (1-\gamma)I_E + \gamma I_F \qquad (0 \leq \gamma = \frac{|EP|}{|EF|} \leq 1).
$$

It is trivial to prove that

$$
\begin{aligned}
P = A &\iff I_P = I_A \\
P = B &\iff I_P = I_B \\
P = C &\iff I_P = I_C \\
P = D &\iff I_P = I_D
\end{aligned}
$$

and thus this algorithm is consisent.

However, Gourard algorithm still has some problems. While interpolation gives shading that is continuous in value, it is not continuous in derivative across boundaries of the polygons, making the edges of the shapes still appear as sharp as they do with flat shading, even though the shape is realistically shaded.

3.2.2 Phong shading

A great improvement on Gouraud shading model came from B.T. Phong and his research on shading (Pho75). Proposing the currently most often used equation of the Blinn-Phong shading model (2.7) he made a great improvement with calculating the specular highlights, however, the research also improved upon the Gourard model.

It proposed that instead of interpolating shading at points, one should interpolate surface normals as follows:

Given a polygon $ABCD$ and a scan line, if the surface normals are known at points A, B, C and D, it is possible to calculate the surface normals at points E and F, the intersections of the scan line with AB and CD respectively, as

$$\begin{aligned} \mathbf{n}_E =& (1-\alpha)\mathbf{n}_A + \alpha\mathbf{n}_B \qquad (0 \leq \alpha \leq 1), \\ \mathbf{n}_F =& (1-\beta)\mathbf{n}_C + \beta\mathbf{n}_D \qquad (0 \leq \beta \leq 1). \end{aligned}$$

.

Since this model makes sure that edges of a polygon have the same surface normal as the edge it shares with neighbouring polygon, it better approximates curvature and with the help of equation (2.7) this model produces a more realistic, though, slightly more complex shading.

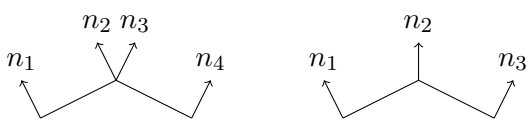


Figure 3.3: Differences between normals in Gouraud (left) and Phong (right) shaders

Nevertheless, both Gouraud and Phong shading models are not suited to deal with rough surfaces, but instead, deal only with smooth textures.

3.2.3 Bump mapping

To portray the shading of a rough surface, such as for example a wall, many programs currently use a technique called *bump mapping* (Bli78). Generating every small irregularity with usual shading algorithms would be very time inefficient and therefore, requires a better approach.

Knowing that shading at point (x, y, z) is dependant on the normal vector at that point, bump mapping uses a perturbed normal with the help of a bump map, i.e. a bivariate scalar function $F(u, v)$.

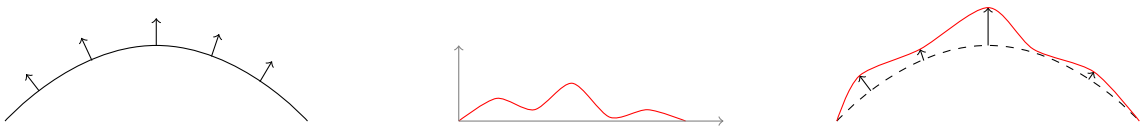


Figure 3.4: Graphical simplification of applying a bump map (red) onto the original surface (black) normals

Namely, it is possible to define any point on a surface with vector

$P = (X(u,v), Y(u,v), Z(u,v))$ where $0 \leq u, v \leq 1$ and it follows that the normal in the point is given as

$$\frac{\partial P}{\partial u} \times \frac{\partial P}{\partial v} = \mathbf{N}$$

since the partial derivatives of P lie on the tangent plane that contains P.

Before calculating shading, the surface normal has to be normalised, thus

$$\mathbf{N}' = \frac{\mathbf{N}}{||\mathbf{N}||}.$$

To create the new displaced point P', one has to apply the bump map function F on the normalised normal $\mathbf{N}$.

Consequently,

$$P' = P + F\frac{\mathbf{N}}{||\mathbf{N}||}.$$

Having displaced the point P', it is now possible to find its normal in a similar way, and apply one of the usual shading algorithms. Accordingly,

$$\mathbf{N}' = \frac{\partial P'}{\partial u} \times \frac{\partial P'}{\partial v}$$

and using the chain rule

$$\begin{aligned}
\frac{\partial P'}{\partial u} =& \frac{\partial P}{\partial u} + \frac{\partial F}{\partial u}\left(\frac{\mathbf{N}}{||\mathbf{N}||}\right) + F\left(\frac{\partial \frac{\mathbf{N}}{||\mathbf{N}||}}{\partial u}\right) \\
\frac{\partial P'}{\partial v} =& \frac{\partial P}{\partial v} + \frac{\partial F}{\partial v}\left(\frac{\mathbf{N}}{||\mathbf{N}||}\right) + F\left(\frac{\partial \frac{\mathbf{N}}{||\mathbf{N}||}}{\partial v}\right).
\end{aligned}$$

As bump mapping was developed to deal with the time inefficiency of generating many small irregularities, it is possible to assume that the value of F is negligable. Hence, the partial derivatives become,

$$\frac{\partial P'}{\partial u} = \frac{\partial P}{\partial u} + \frac{\partial F}{\partial u}\left(\frac{\mathbf{N}}{||\mathbf{N}||}\right), \qquad \frac{\partial P'}{\partial v} = \frac{\partial P}{\partial v} + \frac{\partial F}{\partial v}\left(\frac{\mathbf{N}}{||\mathbf{N}||}\right).$$

Consequently,

$$\mathbf{N}' = \left(\frac{\partial P}{\partial u} + \frac{\partial F}{\partial u}\left(\frac{\mathbf{N}}{||\mathbf{N}||}\right)\right) \times \left(\frac{\partial P}{\partial v} + \frac{\partial F}{\partial v}\left(\frac{\mathbf{N}}{||\mathbf{N}||}\right)\right)$$

or when expanded

$$\mathbf{N}' = \frac{\partial P}{\partial u} \times \frac{\partial P}{\partial v} + \frac{\partial F}{\partial u}\left(\frac{\mathbf{N}}{||\mathbf{N}||} \times \frac{\partial P}{\partial v}\right) + \frac{\partial F}{\partial v}\left(\frac{\partial P}{\partial u} \times \frac{\mathbf{N}}{||\mathbf{N}||}\right) + \frac{\partial F}{\partial u}\frac{\partial F}{\partial v}\left(\frac{\mathbf{N}}{||\mathbf{N}||} \times \frac{\mathbf{N}}{||\mathbf{N}||}\right).$$

From this it is possible to derive the final equation for bump mapping as

$$\mathbf{N}' = \mathbf{N} + \frac{\frac{\partial F}{\partial u}\left(\mathbf{N} \times \frac{\partial P}{\partial v}\right) + \frac{\partial F}{\partial v}\left(\frac{\partial P}{\partial u} \times \mathbf{N}\right)}{||\mathbf{N}||}.$$

3.3 Distances between objects

3.3.1 Closest distance between a moving observer and a moving object

Having derived the formulas for a stationary observer and a moving object it is now quite simple to derive the same for a moving observer as well. With a straightforward change of variables it is possible to classify the observer, the object and their relationship with their respective locations and velocities as:

observer location - p, object location - q, relative object location - $q - p$,

observer velocity - $s_p v_p$, object velocity - sv, relative object velocity - $sv - s_p v_p = s_r v_r$.

Putting these new variables into (2.9) and (2.10) leads to final equations

$$||d|| = ||q + t_c s_r v_r||$$

$$t_C = \frac{-v_r \cdot (q - p)}{s_r},$$

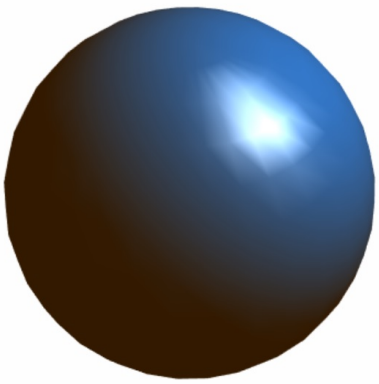
given that the observer's relative location was the origin.

4

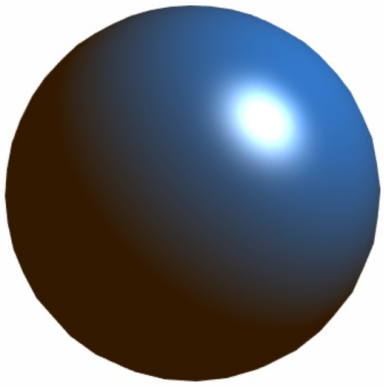
Discussion

Summarizing the results of this thesis, we have now showed the calculations behind the well known perspectives, such as fish-eye and wide-angle lenses, which are being quite used in mobile applications for photo enhancements and by various professional digital artists and photographers.

We have also presented the current shaders being used in digital rendering engines and video games, with an example of both shaders shown in figures 4.1a and 4.1b .



(a) Sphere shaded with Gouraud algorithm



(b) Shere shaded with Phong algorithm

Figure 4.1

While Phong shading algorithm produces more realistic graphics, it does require a longer time to compute, therefore making Gouraurd shader a better option for non-realistic or cartoony games such as Super Mario 64, whereas Phong is being used in more realistic games, such as quite notably, Half-Life 2.

(a) Gouraud shader in Super Mario 64 (©Nintendo, 1996)

(b) Phong shader in Half-Life 2 (©Valve Corporation, 2004)

Figure 4.2

However, with the improvements in technology and GPUs, it has become easier to compute more difficult algorithms and in the last couple of years, we have seen the rise of physically based rendering. Physically based rendering (PBR) uses Monte Carlo integration and estimations, Stochastic progressive photon mapping and bidirectional path tracing to more accurately represent the materials being rendered, as well as their underlying qualities. This rendering makes it possible to represent various imperfections on materials, skin qualities, such as blushing, and transparent materials. Physically based rendering is now being used in most newer RPG (role-playing games) games, which on most part rely on their realism, as well as in newer computer animated movies.

(a) Physically based rendering in Rise of the Tomb Raider (©Crystal Dynamics, 2013)

(b) Physically based rendering in Monsters University (©Pixar Animation Studios, Walt Disney Pictures, 2013)

Figure 4.3

Through all these calculations, hopefully, this thesis manages to uniformly represent algorithms used for creating perspectives, subsequently, incouraging the development

of new perspectives, as well as competently displaying famous shading algorithms in conjunction with their usage.

References

[Bli78] J. Blinn. Simulation of wrinkled surfaces. *Caltech*, 1978. 11, 22

[Gou71] H. Gouraud. Continuous shading of curved surfaces. *IEEE Transactions on computers*, 1971. 20

[KMH95] C. Kolb, D. Mitchell, and P. Hanrahan. A realistic camera model for computer graphics. *ACM*, 1995. 18

[Pho75] B. Phong. Illumination for computer generated pictures. *ACM*, 1975. 10, 21

[Vin07] J. Vince. *Vector Analysis for Computer Graphics*. Springer, 1st edition, 2007. 12

[ZWXH11] T. Zhu, W. Wang, Y. Xie, and P.A. Heng. An ellipsoid-based perspective projection correction on wide-angle images. *SIGGRAPH Asia*, 2011. 19

Lnu.se

Linnæus University
Sweden

Faculty of Technology
SE-391 82 Kalmar | SE-351 95 Växjö
Phone +46 (0)772-28 80 00
teknik@lnu.se
Lnu.se/faculty-of-technology?l=en